

When Is Reuse Safe? Regime-Aware Policy Transfer for Recurring Drift in Network Telemetry

Neha Manoj
Ericsson Research
Bangalore, India
neha.manoj@ericsson.com

Abstract—Network telemetry drifts, and the standard fix, retraining, pays the full cost of rebuilding the model every time. But network regimes recur: a congestion profile, a radio configuration or an operator comes back. We ask whether a policy learned for a regime can be reused when that regime returns, and when doing so is safe. Regime-Aware Policy Transfer (RAPT) stores one ensemble policy per regime, retrieves it on recurrence, and refits it when rolling accuracy shows it no longer fits. On four network streams under a strict prequential protocol, RAPT uses 53–88% less adaptation CPU than full retraining. Macro-F1 is lower on all four streams, significantly so on all four at the seed-level floor. The clearest failure is UGR’16, where regime identifiers recur but the feature-to-label mapping does not. Blind reuse drops macro-F1 to 0.850, and a larger novelty-refit buffer recovers it to 0.899. So reuse keyed on regime identity alone is unsafe, and reuse with a performance check is not. On 5G Campus, a cost-aware refit configuration matches full retraining within measurement noise ($\Delta_{\text{macro-F1}} = -0.0022$, $p = 0.44$) at about 23% less adaptation CPU. Two other mechanisms failed under measurement and we report them: a relative-degradation trigger that cannot fire on a policy that was poor from the start, and an incremental learner that was worse on both accuracy and cost.

Index Terms—concept drift, network telemetry, online learning, adaptive ensembles, regime-aware learning, computationally efficient adaptation

I. INTRODUCTION

The data seen by network monitoring systems does not stay the same for long. Load, radio conditions, subscriber mobility and configuration changes each move the distribution, and a classifier fitted on a period of a 5G testbed or an ISP netflow archive will not stay accurate for very long. Retraining once performance drops is the standard answer. It works, but it has to be paid for again every time, and with high-rate telemetry the total spent on rebuilding models may end up above what it costs to serve predictions. This concern has already led to lightweight detection and adaptation frameworks for constrained devices [27]. Many good detectors exist for deciding *when* to adapt [2], [4], [5], [8], [10], [19], [22], but they make no use of how the change is structured. Conditions in networks often come back. A weekday congestion profile, a radio configuration or an operator-specific QoS regime may go away and return later. Some of the earliest work in the field was driven by recurring concepts [23], [25], and still the usual practice is to learn again from scratch, even though the model being thrown away may well be the right one.

The question we ask here is whether the discarded information can be kept and used. RAPT keeps a repository of adaptation policies, each indexed by an operating regime. At a regime boundary it looks up whether the incoming regime already has a stored policy. When it does, that ensemble is retrieved at close to zero adaptation cost, and when it does not, a new policy is trained and added to the repository. Since a retrieved policy can be wrong if the label relationship in that regime has changed, RAPT also follows its streaming accuracy and refreshes the policy if needed.

If regimes recur with stable label semantics, reuse keeps accuracy and removes most of the adaptation cost. If regimes recur but the feature-to-label relationship has moved, reuse brings back a stale policy and accuracy suffers, and the repair mechanism is the reason the approach still works in that setting. Our contributions are as follows:

- a regime-conditioned policy-transfer mechanism that stores and retrieves ensemble policies by regime;
- a monitoring and refresh strategy whose cost-aware configuration matches full-retraining accuracy at a fraction of the cost;
- an evaluation on four network streams under one prequential protocol, against four established drift detectors;
- a failure analysis covering harmful blind reuse, a trigger that cannot fire by construction, and an incremental learner that loses on both axes.

II. RELATED WORK

A. Concept and Data Drift

Drift in a data stream is usually sorted into two cases. Under covariate shift the marginal $P(X)$ changes while $P(Y | X)$ holds, and under concept drift it is the conditional $P(Y | X)$ that changes [10], [15]. Surveys have catalogued the taxonomies and adaptation strategies built up over the years [13]. The story starts with STAGGER and its recurring concepts [23], and with Widmer’s analysis of robustness against flexibility [25].

B. Drift Detection

ADWIN compares sub-window means inside an adaptive window [4], and Page-Hinkley is a sequential cumulative-sum procedure [19]. DDM and EDD follow the error rate together with its variance [2], [10]. RDDM was designed to give fewer false alarms [3], while the ECDD-EWMA chart

tracks an exponentially weighted moving average of the error [22]. A separate family needs no labels and tests the features directly. Examples use Hoeffding bounds [9], an incremental Kolmogorov–Smirnov statistic [8], or the earth-mover distance [5], and lightweight versions exist for constrained IoT deployments [27].

C. Adaptive and Online Learning

Online ensembles handle drift by updating their members as new data arrives. Dynamic weighted majority weights each learner by its recent accuracy and adds or drops learners along the way [14]; it grew out of earlier methods that minimize disagreement on drifting concepts [12]. Adaptive random forests pair online bagging with drift-aware tree replacement [11], and the accuracy-updated ensemble reweights its components when drift appears [6]. Similar designs have been proposed for social big-data and IoT streams [1], [26]. Single-pass incremental learners, of which Hoeffding trees are the standard example, keep memory use and per-example update cost bounded [7]. The detectors and incremental learners in our experiments come from the River library [18]. All of this work is about *how* a model gets updated. None of it considers fetching a model that is already good enough. In Section V-G we test one incremental learner against our approach, and on our stream it does not improve the accuracy–cost trade-off.

D. Transfer under Recurring Conditions

Transfer learning moves representations or parameters learned on a source task over to a related target task [21]. The idea of holding on to learned structure has a long history, with roots in work on the role of forgetting [16] and on the cost of reusing compiled knowledge chunks [24]. It saves a fitted policy with its regime identity and retrieves the two together. The decision it makes is whether any adaptation is needed, and how to adapt is left aside. With no recurrence, the mechanism collapses into ordinary retraining.

III. PROBLEM FORMULATION

We treat the data as a chronological stream $x_1, x_2, \dots$, where $x_t \in \mathbb{R}^d$ is a feature vector and $y_t \in \{1, \dots, C\}$ is its label. The label is unknown until the prediction for x_t has been issued. The stream is divided into fixed-size windows $W_1, \dots, W_T$, and each window W_k is tagged with a regime identifier $r_k \in \mathcal{R}$. A regime is a semantically defined operating condition, such as a network scenario or a time-of-day block. It is not a cluster label. A *policy* π is a fitted model plus its decision rule, and the repository is a mapping $\mathcal{P} : \mathcal{R} \rightarrow \pi$ that is filled in as new regimes show up. Whenever the stream moves from r_{k-1} to r_k , the controller picks an action $a_k \in \{\text{REUSE}, \text{TRAIN}\}$. It reuses $\mathcal{P}(r_k)$ when that exists, and trains a new policy on the recent buffer when it does not. Let $M(\pi, W)$ be a performance measure, which is macro-F1 in this paper, and let $c(\pi)$ be the cost of producing π in adaptation CPU time. The objective is to hold $M(\cdot, W_k)$ near the level a freshly trained policy would reach while making $\sum_k c(a_k)$ as small as possible.

Reuse rests on an assumption, namely that $P(Y \mid X, r_k)$ has not changed since the policy was stored. If the regime identifier returns after the conditional relationship has shifted, the stale policy gets applied and nothing flags it. The hard part is separating a recurring regime from a recurring label relationship.

IV. PROPOSED METHOD: RAPT

A. Architecture and Repository

RAPT sits between the stream and the model and makes one decision at each regime boundary. The active policy predicts each window, and the window is scored when its labels arrive. A change of policy happens only at a regime transition or when monitoring reports degradation.

We represent a regime with its semantic identifier and do not learn a clustering. What we give up is that the mechanism requires a usable regime signal; see Section VII. The repository holds one checkpoint per regime, made up of the fitted ensemble, its soft-voting weights, and timestamps for creation and last access. Checkpoints are immutable.

B. Matching and Transfer

Matching uses the regime identifier. On a transition to regime r , the controller tests whether $r \in \mathcal{P}$. The enhanced variant also applies a fingerprint-similarity gate, which requires the feature summary of the current window to be close to the fingerprint saved when the checkpoint was created. The effect is to relax the question “has this regime been seen?” into “has a sufficiently similar instance been seen?”

When reuse goes ahead, the retrieved ensemble becomes the active policy, and its soft-voting weights can be recalibrated using the most recent buffered window. Calibration computes the recent accuracy of the two components and renormalizes their weights. It covers at most the last 50 observations and is skipped when fewer than 20 are available, so its cost is small and fixed. If no checkpoint exists, we train a new policy from a class-anchored buffer and store it. The buffer holds a stratified slice of the training prefix. All adaptive models here use it in the same way, which stops a refit from collapsing onto a single class when the recent window has only one.

C. Monitoring, Refresh, and Fallback

We use two monitoring rules, and how they differ is one of our findings. The *absolute* rule triggers when rolling accuracy goes under a fixed floor. The *relative* rule triggers when accuracy has dropped by more than a fixed margin from a decayed reference, which is built from that policy’s own earlier accuracy. A relative rule cannot catch a policy that was never good, because the reference decays toward the poor level and so the gap it looks for never appears.

A refresh rebuilds the active policy from the recent buffer and replaces its checkpoint, and any later reuse then gets the refreshed policy. In the default refresh, the 100-tree ensemble is refit on a 1000-observation buffer. The cost-aware refresh has the same trigger but refits 20 trees on a 300-observation buffer. RAPT-Enhanced contains two additions. *Buffer-blended*

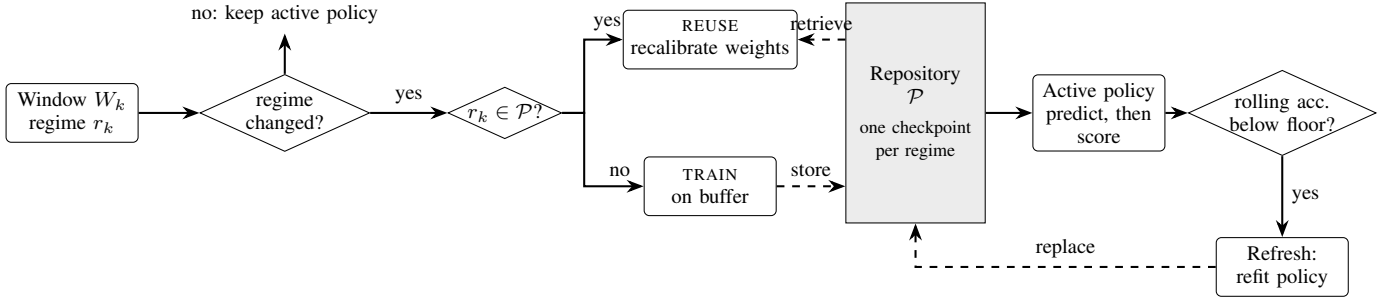


Fig. 1. RAPT overview. At each regime boundary the controller looks up the incoming regime in the repository and either reuses the stored policy or trains and stores a new one. The active policy predicts and is scored as labels arrive. A monitor refits the policy and replaces its checkpoint when rolling accuracy degrades. Solid arrows are control flow, dashed arrows are repository reads and writes.

novelty refitting refits a novel regime on a 1500-observation buffer, where the original used 500. *Selective parity refitting* refits a reused policy and replaces its checkpoint whenever rolling accuracy falls under a parity threshold over a short window. This targets stale reuse directly. When there is no checkpoint, or the similarity gate turns down the one that exists, the controller trains a fresh policy and stores it under the incoming regime identifier.

V. EXPERIMENTAL METHODOLOGY

A. Datasets

There are four datasets summarized in Tab.I, all of which contain data regarding Quality Of Service (QoS) in networks. All of these are based upon real-world network traces. The data in this paper were obtained from a number of sources. Data from a campus-based testbed is represented in “5G Campus Network QoS” and includes both delay metrics and classifications. This classification consists of three categories, GOOD, DEGRADED, and BAD. In order to determine whether the delay is classified as GOOD, DEGRADED, or BAD the 90th percentile value of the delay is determined in the next window. Each of the three scenarios will occur fourteen times. The second data set “UGR’16,” represents aggregated netflow data from an ISP. There are one hundred thirty-three features in the UGR’16 dataset. Due to the fact that there is very little labeled attack traffic in the calibration portion of the data (only blacklisted traffic), we chose to utilize the first thirty days of the labeled data rather than the calibration split. Since there is one labeled sample per minute, the total number of samples in the labeled data is forty-three thousand two hundred. Time-of-day and weekday/weekend, when combined, results in twelve unique regimes and 168 unique recurrences. “NordicDat” is comprised of cross border LTE and 5G traces to be utilized for predictive quality of service [17]. Both of the above mentioned targets consist of three classes and are based upon the median delay of the next window. The regimes within “NordicDat” are based upon mobile operator. Finally, “5G NR End-to-End Latency” is a simulated dataset [20] of 499 windows with 500 packets each, distributed across four regimes (ABCDACBDAB). Similar to “Campus,” the target

for “5G NR” utilizes the same target metric as the campus stream.

TABLE I
DATASETS USED IN EVALUATIONS. RECURRENCES DENOTE OCCURRENCES WHERE A PARTICULAR REGIME IS REVISITED.

Property	Campus	UGR’16	Nordic	5G NR
Samples	1,790	43,200	91,000	499
Features	13	133	15	11
Classes	3	2	3	3
Windows	179	180	182	499
Window size	10	240	500	500
Regimes	3	12	3	4
Recurrences	14	168	17	6

B. Preprocessing, Ordering, and Leakage Control

All four streams undergo the exact same processing. First, all four streams undergo sorting by row according to times-tamp. Then, all identifier and constant columns are dropped. Finally, only those features that are numeric are retained. Any infinite values present within a stream are treated as missing. Any rows that lack a target value are deleted. Any remaining gaps are then filled using medians computed solely from the training prefix. Target thresholds and any scale-related statistics also derive from that prefix. None of the streams are shuffled. The prefix is the first twenty percent of windows: thirty-six for the three cross-dataset streams and one hundred for 5G NR. That prefix is completely excluded from evaluation, and that is the sole source of target thresholds. The order of operations is predict → evaluate → adapt for every window prior to its labels being used for adaptation. Regime annotation occurs directly from reading the stream. Therefore, no derivation of annotation can occur based upon the target.

C. Models And Detectors

Each of the five primary models utilizes exactly the same base architecture and the same retraining buffer. Therefore, any variation between them is due to their respective adaptation mechanisms only. The base is an ensemble of a random forest classifier and an extra-trees classifier, both having fifty trees of depth seven. Frozen is trained once on the prefix

and receives no subsequent updates. Event-Driven will update when window accuracy falls below a fixed threshold. Full Retraining will recreate/rebuild the model at every regime transition. RAPT and RAPT-Enhanced represent the controller(s) explained in Section IV.

ADWIN, Page-Hinkley, EDDM and ECDD-EWMA operate within the same retraining harness, with identical model, buffer, capacity and minimum-retrain-interval, differing only in how they detect triggers. Parameters were fixed in advance and not tuned against the test streams (ADWIN $\delta = 0.002$; Page-Hinkley $\min = 30$, $\delta = 0.005$; EDDM $\alpha = 0.95$, $\beta = 0.9$; ECDD-EWMA $\lambda = 0.2$, $L = 2$). Since none of our streams include true ground truth drift boundary indicators we report detected events, adaptation costs and predictive performance. Precision and Recall were not reported for our detectors.

D. Metrics And Statistics

Performance is evaluated through macro-F1, Accuracy, Precision and Recall measures. Costs may be measured in two fashions. Adaptation CPU denotes CPU time exclusively devoted to adaptation routines while Runtime indicates Total Streaming Time. A reduction in Adaptation Time does not necessarily translate into a proportional reduction in Total Runtime.

Each experiment had seeds [42,43,44,45,46]. Therefore we report Mean and Standard Deviation for each measure. For comparison purposes we use the Wilcoxon Signed-Ranked Test for paired comparisons along with Cohen's d. With respect to pairing by Window there would be approximately 143–146 observations. When paired by Seed we would have $n = 5$ with a minimum two-sided p-value available to us of .0625. If a result shows no significance we state that no difference was demonstrated between methods.

E. Predictive Performance

Table II holds the primary comparison, and Fig. 2 shows the macro-F1 values side by side.

On 5G Campus every model is close to saturation. Full Retraining has the best macro-F1 (0.9739 ± 0.0049), while RAPT and RAPT-Enhanced both land at 0.9402 ± 0.0028 . That is 0.0338 lower, and the gap is significant ($p = 0.0625$, $d_z = -13.05$).

UGR'16 is where the assumptions of the mechanism fail, and it teaches us the most. Frozen has the highest macro-F1 (0.9688 ± 0.0038), which is consistent with attacks being sparse and clustered. RAPT reaches only 0.8503 ± 0.0346 . This is 0.1088 below Full Retraining ($p = 0.0625$, $d_z = -3.16$) and the worst of the five models, despite its 132 reuses. The twelve regimes do recur, and their identifiers stay valid throughout the 30-day block, but the mapping from netflow features to attack labels does not stay put. When RAPT-Enhanced refits a reused policy that has degraded, macro-F1 recovers to 0.8988 ± 0.0412 . The gain of 0.0485 ($p = 0.0625$, $d_z = 1.05$) is the one statistically supported improvement of a variant over another on this stream; an ablation attributes it

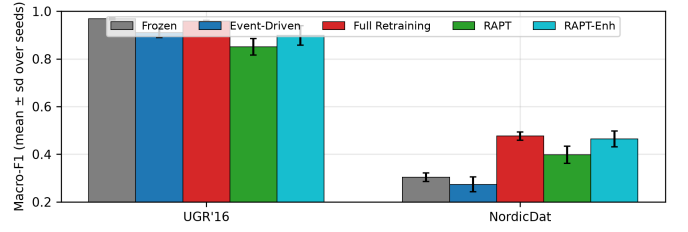


Fig. 2. Macro-F1 on the two largest cross-dataset streams. On UGR'16, RAPT tracks the frozen baseline and not the retrained models, which is the signature of stale policy reuse. On the 5G NR stream the five models span 0.1103.

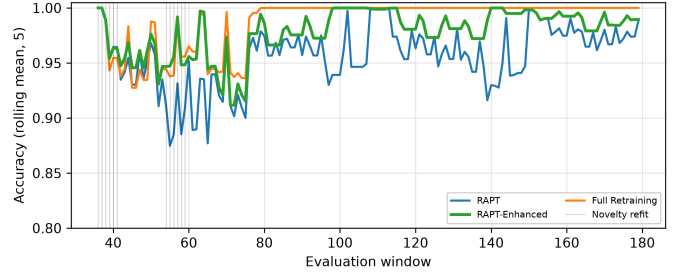


Fig. 3. Rolling accuracy over evaluation windows on UGR'16. RAPT sags once reused policies go stale, and the larger novelty-refit buffer in RAPT-Enhanced pulls accuracy back toward Full Retraining.

to the larger novelty-refit buffer (500 $\rightarrow$ 1500 observations), not to the parity trigger, which fires zero times at its default threshold.

NordicDat is hard to predict. Frozen scores 0.3037 ± 0.0182 , adaptation helps, and Full Retraining gets 0.4765 ± 0.0171 against 0.3978 ± 0.0361 for RAPT. The deficit of 0.0786 is significant at the seed level ($p = 0.0625$, $d_z = -4.04$). RAPT does have the highest accuracy on this stream (0.6037 ± 0.0099 against 0.5920 ± 0.0077). On 5G NR the five models span 0.1103. Taken together, the predictive results give no basis for saying that RAPT improves accuracy. Full Retraining has the best macro-F1 on two of the four streams, and RAPT sits below it on all four. The case for RAPT depends on cost, and on knowing when reuse is safe.

F. Computational Cost and Policy Reuse

Table II also lists adaptation and total cost, and Fig. 4 compares adaptation CPU. Relative to Full Retraining, the cost falls on every stream: from 1.2779 to 0.2568 s on 5G Campus (−79.9%), from 23.6137 to 2.7376 s on UGR'16 (−88.4%), from 2.1731 to 0.3146 s on NordicDat (−85.5%), and from 0.6070 to 0.2901 s on 5G NR (−52.2%). Full Retraining retrains at every regime transition, meaning 13, 143, 18 and 7 times. RAPT retrains 2, 11, 2 and 3 times and retrieves a stored policy 11, 132, 16 and 4 times.

G. Efficiency Ablation

Table III and Fig. 6 show the mechanism ladder on 5G Campus, each row adding one mechanism. The base controller (RAPT-Full) scores 0.9537 ± 0.0064 macro-F1 at

TABLE II

PRIMARY COMPARISON AND COST, MEAN OVER SEEDS 42–46. BOLD MARKS THE BEST MACRO-F1 PER STREAM; CPU AND RUNTIME IN SECONDS, EVENTS ARE COUNTS.

Dataset	Model	F1	Acc.	Prec.	Rec.	CPU (s)	Run (s)	Retr.	Reuse
5G Campus	Frozen	0.9370 $\pm$ 0.0010	0.9822 $\pm$ 0.0006	0.9386 $\pm$ 0.0010	0.9383 $\pm$ 0.0003	0.0000	1.2307	0	–
	Event-Driven	0.9610 $\pm$ 0.0030	0.9898 $\pm$ 0.0018	0.9609 $\pm$ 0.0033	0.9627 $\pm$ 0.0019	0.1190	1.3700	1	–
	Full Retraining	0.9739 $\pm$ 0.0049	0.9929 $\pm$ 0.0013	0.9741 $\pm$ 0.0044	0.9751 $\pm$ 0.0049	1.2779	2.5167	13	–
	RAPT	0.9402 $\pm$ 0.0028	0.9831 $\pm$ 0.0017	0.9421 $\pm$ 0.0012	0.9414 $\pm$ 0.0032	0.2568	1.4766	2	11
	RAPT-Enhanced	0.9402 $\pm$ 0.0028	0.9831 $\pm$ 0.0017	0.9421 $\pm$ 0.0012	0.9414 $\pm$ 0.0032	0.2628	1.5065	2	11
UGR'16	Frozen	0.9688 $\pm$ 0.0038	0.9905 $\pm$ 0.0006	0.9784 $\pm$ 0.0034	0.9661 $\pm$ 0.0037	0.0000	1.3203	0	–
	Event-Driven	0.9104 $\pm$ 0.0211	0.9899 $\pm$ 0.0019	0.9177 $\pm$ 0.0205	0.9081 $\pm$ 0.0217	0.6537	2.0142	3	–
	Full Retraining	0.9591 $\pm$ 0.0031	0.9877 $\pm$ 0.0010	0.9722 $\pm$ 0.0022	0.9576 $\pm$ 0.0027	23.6137	25.0668	143	–
	RAPT	0.8503 $\pm$ 0.0346	0.9565 $\pm$ 0.0208	0.8688 $\pm$ 0.0321	0.8491 $\pm$ 0.0354	2.7376	4.0972	11	132
	RAPT-Enhanced	0.8988 $\pm$ 0.0412	0.9781 $\pm$ 0.0205	0.9131 $\pm$ 0.0375	0.8972 $\pm$ 0.0402	3.2996	4.6586	11	132
NordicDat	Frozen	0.3037 $\pm$ 0.0182	0.4843 $\pm$ 0.0074	0.4479 $\pm$ 0.0183	0.2767 $\pm$ 0.0191	0.0000	1.4280	0	–
	Event-Driven	0.2735 $\pm$ 0.0309	0.4620 $\pm$ 0.0449	0.4059 $\pm$ 0.0186	0.2476 $\pm$ 0.0273	0.2282	1.6855	2	–
	Full Retraining	0.4765 $\pm$ 0.0171	0.5920 $\pm$ 0.0077	0.5498 $\pm$ 0.0159	0.4629 $\pm$ 0.0180	2.1731	3.6260	18	–
	RAPT	0.3978 $\pm$ 0.0361	0.6037 $\pm$ 0.0099	0.4863 $\pm$ 0.0328	0.3767 $\pm$ 0.0372	0.3146	1.7711	2	16
	RAPT-Enhanced	0.4644 $\pm$ 0.0332	0.5996 $\pm$ 0.0120	0.5411 $\pm$ 0.0272	0.4494 $\pm$ 0.0343	1.4094	2.8665	2	10
5G NR Lat.	Frozen	0.9048 $\pm$ 0.0056	0.9048 $\pm$ 0.0056	0.9048 $\pm$ 0.0056	0.9048 $\pm$ 0.0056	0.0000	3.4044	0	–
	Event-Driven	0.8576 $\pm$ 0.0045	0.8576 $\pm$ 0.0045	0.8576 $\pm$ 0.0045	0.8576 $\pm$ 0.0045	1.1178	4.5229	13	–
	Full Retraining	0.8586 $\pm$ 0.0074	0.8586 $\pm$ 0.0074	0.8586 $\pm$ 0.0074	0.8586 $\pm$ 0.0074	0.6070	4.0174	7	–
	RAPT	0.7945 $\pm$ 0.0163	0.7945 $\pm$ 0.0163	0.7945 $\pm$ 0.0163	0.7945 $\pm$ 0.0163	0.2901	3.7132	3	4
	RAPT-Enhanced	0.8446 $\pm$ 0.0106	0.8446 $\pm$ 0.0106	0.8446 $\pm$ 0.0106	0.8446 $\pm$ 0.0106	0.5998	4.0036	3	4

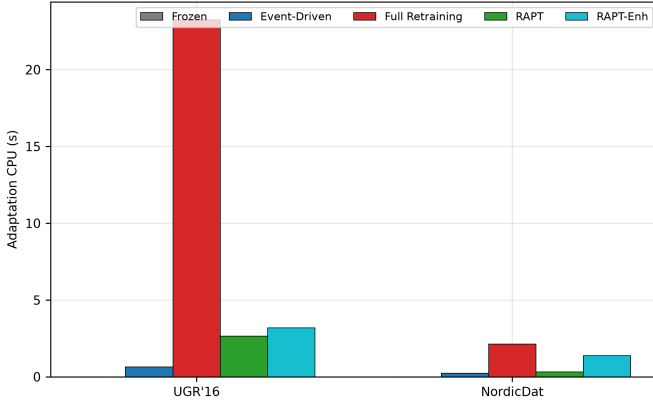


Fig. 4. Adaptation CPU on the two largest streams. The ordering is the reverse of Fig. 2: on UGR'16, RAPT uses an order of magnitude less adaptation time than Full Retraining and records lower macro-F1.

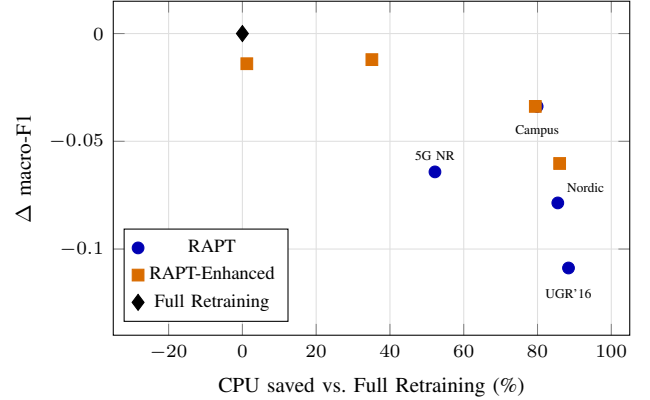


Fig. 5. Accuracy–cost trade-off on all four streams, measured against Full Retraining at the origin. Circles are RAPT and squares are RAPT-Enhanced, computed from Table II. Up and to the right is better. Squares left of zero cost more adaptation CPU than Full Retraining.

0.6211 s. Replacing its trigger with a relative-degradation trigger (RAPT-Evidence) fires *zero* refreshes and leaves macro-F1 at 0.9402 ± 0.0028 , as does an absolute floor (RAPT-Floor), because Campus window accuracy never falls below the 0.60 floor. The periodic schedule is what refreshes: a full refit every five windows scores 0.9745 ± 0.0044 at 4.6486 s (22 refreshes). RAPT-Cheap keeps that schedule with the cheaper refit, scoring 0.9717 ± 0.0081 at 0.9801 s against Full Retraining's 0.9739 ± 0.0049 at 1.2779 s ($\Delta = -0.0022$, $p = 0.4375$, $d_z = -0.34$, not significant), matching accuracy while cutting adaptation CPU by roughly 79%. RAPT-Incremental (0.7164 ± 0.0057 at 0.1874 s) is dominated on both axes. The evidence supports matching accuracy at greatly reduced cost, not better performance.

TABLE III

PERFORMANCE OF DIFFERENT CONFIGURATIONS OF RAPT USING THE 5G CAMPUS STREAM, AVERAGED OVER MULTIPLE RANDOM SEEDS.

Configuration	Macro-F1	Adapt. CPU	Refreshes
Full Retraining. (ref.)	0.9739 $\pm$ 0.0049	1.2779	–
RAPT-Full	0.9537 $\pm$ 0.0064	0.6211	0
RAPT-Evidence	0.9402 $\pm$ 0.0028	0.2582	0
RAPT-Floor	0.9402 $\pm$ 0.0028	0.2568	0
RAPT-Periodic-FullRefit	0.9745 $\pm$ 0.0044	4.6486	22
RAPT-Cheap	0.9717 $\pm$ 0.0081	0.9801	22
RAPT-Incremental	0.7164 $\pm$ 0.0057	0.1874	130

H. Determinants of Cost and Trigger Behavior

We chose the cheap refresh from direct measurement. Holding the ensemble fixed and varying one factor at a time, a 20-tree refit on a 1000-observation buffer cost 65.8 ms against 320.1 ms for 100 trees, while holding tree count at 20 and

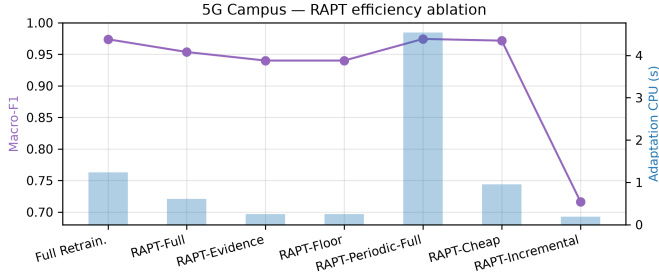


Fig. 6. Ablation ladder for the RAPT configurations on 5G Campus. RAPT-Cheap matches the full retrainer’s macro-F1 at about 23% less adaptation CPU; RAPT-Incremental is worse on both axes.

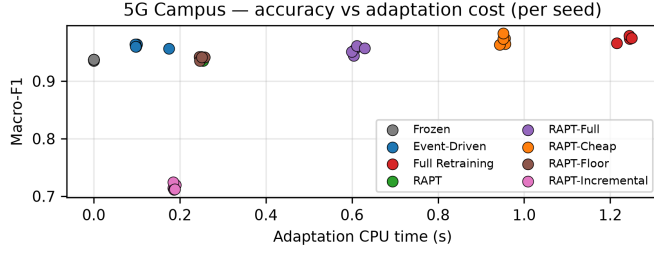


Fig. 7. Accuracy–cost view of the RAPT variants on 5G Campus. RAPT-Cheap reaches the full retrainer’s macro-F1 at lower adaptation CPU, while conservative triggers coincide with the frozen baseline on both axes.

growing the buffer from 200 to 1000 changed the cost from only 37.3 to 65.8ms. Tree count, not buffer size, dominates the refit cost.

RAPT-Evidence generates no refreshes (Section V-G) for a mechanism-level reason. A relative trigger compares current accuracy against a decayed reference built from that policy’s own past, so when a regime is persistently poor rather than deteriorating the reference decays toward the poor level, the gap the rule tests for never opens, and the trigger stays silent though the policy is unfit. On 5G Campus neither rule fires, because window accuracy never drops below the floor; on NordicDat and 5G NR both fire, the relative rule more often (31.4/43.4 refreshes against 28.2/15.6). The floor threshold is stream-specific.

I. Comparison to Historical Detectors

Under the shared harness all four detectors use the same per-window error signal as Event-Driven, with identical model, buffer and retrain throttle. On that coarse signal (143–146 windows for the cross-dataset streams) each fires at most twice: ADWIN, Page-Hinkley and ECDD-EWMA fire 0 times on all four streams and reproduce the frozen baseline exactly, while EDDM fires 0/0/2/1 (Campus/UGR’16/NordicDat/5G NR). The near-zero counts are a signal-scale artefact, not detector inertness: fed their native per-sample error bits, ADWIN fires 0/12/164/0, Page-Hinkley 0/2/103/0 and EDDM 0/8/111/1.

RAPT is the only method here that adapts on every stream while spending well under the full-retrainer budget. It fired 2/11/2/3 adaptation events at 0.2568, 2.7376, 0.3146

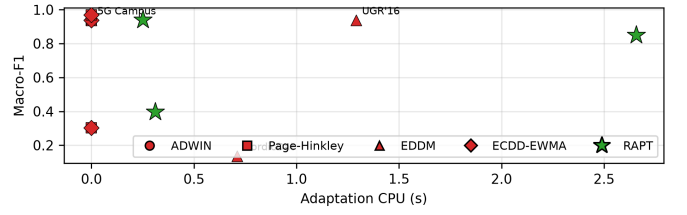


Fig. 8. Historical detectors. On the shared window-error signal ADWIN, Page-Hinkley and ECDD-EWMA never fire and replicate the frozen baseline exactly; EDDM fires at most once. RAPT adapts on every stream, paying an accuracy floor relative to the retrainer.

and 0.2901 s; its adaptation-CPU saving over Full Retraining ranges from 52% (5G NR) to 89% (UGR’16), and it was more accurate than the (non-adapting) detector baselines on 5G NR only. As Fig. 8 shows, RAPT is the least-adapting method that adapts, and its accuracy floor relative to the retrainer is the price of that reuse.

VI. DISCUSSION

The cost saving appears wherever reuse occurs, but the accuracy penalty concentrates on UGR’16, where regimes recur while their label relationship does not, so a reused policy is stale. The repair came from a larger novelty-refit buffer (500 $\rightarrow$ 1500 observations), not the parity trigger, lifting RAPT from 0.8503 to 0.8988, so reuse should carry a check on the reused policy’s recent performance: the regime identifier alone is unsafe when label semantics can drift independently.

VII. LIMITATIONS AND THREATS TO VALIDITY

With five seeds the smallest attainable seed-level p -value is 0.0625, so some large effects cannot reach formal significance, most visibly in the ablation; window-level tests treat same-stream windows as independent and so support direction rather than generalisation. The mechanism relies on recurrence, which we measure but do not assume, and a novel regime falls back to retraining with no advantage. The RAPT-Floor threshold and the 20-tree refit were tuned on the evaluated streams and would need revisiting where the pre/post-drift gap is larger, and regime similarity is taken as meaningful, which may fail if the regime signal is noisy.

VIII. CONCLUSION

Retraining is the default answer to drift, but when conditions recur much of that cost is avoidable. RAPT stores each regime’s policy and reloads it when the regime returns, retraining only when none suits and refreshing a stored policy once monitoring shows it no longer fits. Across four streams, adaptation CPU fell 52–88% relative to full retraining, at the cost of a lower macro-F1 on every stream; where a regime recurred with a changed label relationship, blind reuse caused substantial harm that a larger refit buffer repaired. Policy reuse is deployable where regimes recur with consistent or monitored semantics, and a natural next step is to test a retrieved policy for semantic agreement before reuse.

REFERENCES

- [1] Ahmad Abbasi, Abdul Rehman Javed, Chinmay Chakraborty, Jamel Nebhen, Wishu Zehra, and Zunera Jalil. Elstream: An ensemble learning approach for concept drift detection in dynamic social big data stream learning. *IEEE Access*, 9:66408–66419, 2021.
- [2] Manuel Baena-García, José del Campo-Ávila, Raul Fidalgo, Albert Bifet, Ricard Gavaldà, and Rafael Morales-Bueno. Early drift detection method. In *Proc. Int. Workshop on Knowledge Discovery from Data Streams*, pages 77–86, 2006.
- [3] Roberto S. M. Barros, Danilo R. L. Cabral, Paulo M. Gonçalves, and Silas G. T. C. Santos. Rddm: Reactive drift detection method. *Expert Systems with Applications*, 90:344–355, 2017.
- [4] Albert Bifet and Ricard Gavaldà. Learning from time-changing data with adaptive windowing. In *Proc. SIAM Int. Conf. Data Mining (SDM)*, pages 443–448, 2007.
- [5] Tobias Brockhoff, Merih Seran Uysal, and Wil M. P. van der Aalst. Time-aware concept drift detection using the earth mover’s distance. In *Proc. Int. Conf. Process Mining (ICPM)*, pages 33–40, 2020.
- [6] Dariusz Brzeziński and Jerzy Stefanowski. Reacting to different types of concept drift: The accuracy updated ensemble algorithm. *IEEE Transactions on Neural Networks and Learning Systems*, 25(1):81–94, 2014.
- [7] Pedro Domingos and Geoff Hulten. Mining high-speed data streams. In *Proc. ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, pages 71–80, 2000.
- [8] Denis Moreira dos Reis, Peter Flach, Stan Matwin, and Gustavo Batista. Fast unsupervised online drift detection using incremental kolmogorov–smirnov test. In *Proc. ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, pages 1545–1554, 2016.
- [9] Isvani Frias-Blanco, José del Campo-Ávila, Gonzalo Ramos-Jimenez, Rafael Morales-Bueno, Agustín Ortiz-Díaz, and Yailé Caballero-Mota. Online and non-parametric drift detection methods based on hoeffding’s bounds. *IEEE Transactions on Knowledge and Data Engineering*, 27(3):810–823, 2015.
- [10] Joao Gama, Pedro Medas, Gladys Castillo, and Pedro Rodrigues. Learning with drift detection. In *Proc. Brazilian Symp. Artificial Intelligence (SBIA)*, pages 286–295, 2004.
- [11] Heitor M. Gomes, Albert Bifet, Jesse Read, Jean Paul Barddal, Fabrício Enembreck, Bernhard Pfahringer, Geoff Holmes, and Talel Abdesslem. Adaptive random forests for evolving data stream classification. *Machine Learning*, 106(9–10):1469–1495, 2017.
- [12] David P. Helmbold and Philip M. Long. Tracking drifting concepts by minimizing disagreements. *Machine Learning*, 14(1):27–45, 1994.
- [13] Imen Khamassi, Moamar Sayed-Mouchaweh, Moez Hammami, and Khaled Ghédira. Discussion and review on evolving data streams and concept drift adapting. *Evolving Systems*, 9(1):1–23, 2018.
- [14] J. Zico Kolter and Marcus A. Maloof. Dynamic weighted majority: An ensemble method for drifting concepts. *Journal of Machine Learning Research*, 8(91):2755–2790, 2007.
- [15] Jie Lu, Anjin Liu, Fan Dong, Feng Gu, Joao Gama, and Guangquan Zhang. Learning under concept drift: A review. *IEEE Transactions on Knowledge and Data Engineering*, 31(12):2346–2363, 2018.
- [16] Shaul Markovitch and Paul D. Scott. The role of forgetting in learning. In *Machine Learning Proceedings 1988*, pages 459–465. Morgan Kaufmann, 1988.
- [17] Topi Miekala, Pasi Pykönen, Georgios Drainakis, Panagiotis Pantazopoulos, Tobias Müller, Konstantinos V. Katsaros, Vasilis Sourlas, Angelos Amditis, and Dimitra I. Kaklamani. Nordidat: A cross-border predictive qos dataset. In *Proc. IEEE Global Communications Conf. (GLOBECOM)*, pages 1281–1286, 2024. Dataset: Zenodo 10.5281/zenodo.10964584, CC-BY-4.0.
- [18] Jacob Montiel, Max Halford, Saulo Martiello Mastelini, Geoffrey Bolmier, Raphael Sourty, Robin Vaysse, Adil Zouitine, Heitor Murilo Gomes, Jesse Read, Talel Abdesslem, and Albert Bifet. River: machine learning for streaming data in python. *Journal of Machine Learning Research*, 22(110):1–8, 2021.
- [19] Ewan S. Page. Continuous inspection schemes. *Biometrika*, 41(1/2):100–115, 1954.
- [20] Marcin Pakula and Paweł Kryszkiewicz. 5g nr end-to-end latency simulation dataset. Zenodo record 20035549, CC-BY-4.0, 2026.
- [21] Sinno Jialin Pan and Qiang Yang. A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10):1345–1359, 2010.
- [22] Gordon J. Ross, Niall M. Adams, Dimitris K. Tasoulis, and David J. Hand. Exponentially weighted moving average charts for detecting concept drift. *Pattern Recognition Letters*, 33(2):191–198, 2012.
- [23] Jeffrey C. Schlimmer and Richard H. Granger. Incremental learning from noisy data. *Machine Learning*, 1(3):317–354, 1986.
- [24] Milind Tambe and Allen Newell. Some chunks are expensive. In *Machine Learning Proceedings 1988*, pages 451–458. Morgan Kaufmann, 1988.
- [25] Gerhard Widmer. Combining robustness and flexibility in learning drifting concepts. In *Proc. European Conf. Artificial Intelligence (ECAI)*, pages 468–472, 1994.
- [26] Li Yang, Dimitrios Michael Manias, and Abdallah Shami. Pwpae: An ensemble framework for concept drift adaptation in iot data streams. In *Proc. IEEE Global Communications Conf. (GLOBECOM)*, pages 1–6, 2021.
- [27] Li Yang and Abdallah Shami. A lightweight concept drift detection and adaptation framework for iot data streams. *IEEE Internet of Things Magazine*, 4(2):96–101, 2021.